# Threshold Points in Matrices: Existence, Uniqueness, and Algorithms

Kavindra Mohan Dwivedi[a], Kanhaiya Kumar Jaiswal[a], Varunkumar Jayapaul[a,*]

[a]*School of Computing and Electrical Engineering, Indian Institute of Technology Mandi, Mandi 175005, Himachal Pradesh, India*

**Abstract**

We introduce the *threshold point*, a new matrix parameter arising from two traversal procedures previously used by Dallant et al. [1] for strict saddlepoint detection. We define the threshold point as the value in the matrix for which both these types of traversals end in success.

Our main result is that every matrix contains exactly one threshold-point value. Thus, unlike strict saddlepoints, which may not exist, threshold points are guaranteed to exist and are unique by value. Furthermore, threshold points extend strict saddlepoints in the following sense: whenever a strict saddlepoint exists, its value is equal to the value of the threshold point. We also relate threshold points to the pseudo saddlepoints introduced by Dallant et al., showing that although a matrix may contain multiple pseudo-saddlepoint values, every matrix contains exactly one threshold-point value.

We establish several structural properties of threshold points and present a non-trivial $O(n^{1.5}\sqrt{\lg n})$ algorithm for finding the threshold point of an $n \times n$ matrix.

*Keywords:* saddlepoint, threshold points, comparison-based algorithms, matrix algorithms

*Corresponding author

*Email addresses:* s24004@students.iitmandi.ac.in (Kavindra Mohan Dwivedi), s24086@students.iitmandi.ac.in (Kanhaiya Kumar Jaiswal), varunkumar@iitmandi.ac.in (Varunkumar Jayapaul)

## 1. Introduction

Saddlepoints are a classical concept that arise in several areas of mathematics, optimization, game theory, and computer science. In this paper, we study saddlepoints in matrices and introduce the *threshold point*, a new matrix parameter derived from traversal procedures originally developed for strict saddlepoint detection. These traversals reveal a structural phenomenon that extends beyond the existence of strict saddlepoints and naturally induces a matrix parameter that exists for every matrix. Unlike saddlepoints and strict saddlepoints, this parameter is guaranteed to exist and is unique by value.

A saddlepoint of an $m \times n$ matrix $M$ is an entry $M[i][j]$ that is a maximum in its row and a minimum in its column. A strict saddlepoint is an entry that is the unique maximum in its row and the unique minimum in its column. A saddlepoint value may occur at multiple locations in a matrix, but all saddlepoints, whenever they exist, share the same value. In contrast, a matrix can contain at most one strict saddlepoint. Consequently, algorithms for saddlepoints typically focus on identifying the saddlepoint value, whereas algorithms for strict saddlepoints focus on locating the unique strict saddlepoint. Neither saddlepoints nor strict saddlepoints are guaranteed to exist in every matrix.

The primary objective of this paper is to establish the existence and uniqueness of the proposed parameter, investigate its structural properties, relate it to pseudo saddlepoints and strict saddlepoints, and develop efficient algorithms for computing it.

The saddlepoint problem has a long history in computer science. Knuth [2] presented several $O(n^2)$ algorithms for finding all saddlepoints in an $n \times n$ matrix. Llewellyn et al. [3] later proved that this bound is optimal in the worst case and gave an $O(n^{\lg 3})$ algorithm for saddlepoint detection. Bienstock et al. [4] subsequently proposed an algorithm that probes only $O(n)$ matrix entries and runs in $O(n \lg n)$ time. After nearly three decades, Dallant et al. [1] eliminated the logarithmic overhead introduced by the heap data structure and obtained a deterministic $O(n \lg^* n)$ algorithm. Throughout this paper, we refer to the paper by Dallant et al. [1] as the DD paper to distinguish it from the later randomized $O(n)$ algorithm of Dallant et al. [5].

One of the key ideas introduced in the DD paper is the notion of a *pseudo saddlepoint*: a value $x$ such that every row contains an entry at least $x$ and every column contains an entry at most $x$. The DD paper proved that every

matrix contains at least one pseudo saddlepoint, although multiple distinct pseudo-saddlepoint values may exist. Moreover, whenever a saddlepoint exists, every pseudo saddlepoint has the same value as the saddlepoint. We show that the traversal procedure underlying the DD paper naturally identifies a distinguished pseudo saddlepoint that is unique by value; this is precisely the threshold point.

Several probabilistic results concerning saddlepoints are also known [6, 7].

### *1.1. Our Contributions*

The main contributions of this paper are as follows.

- We introduce the threshold point, a new matrix parameter derived from the traversal procedures used in strict saddlepoint detection.
- We prove that every matrix contains exactly one threshold point value.
- We establish that whenever a strict saddlepoint exists, its value is equal to the threshold point value.
- We develop several structural properties of the proposed parameter and relate it to pseudo saddlepoints and strict saddlepoints.
- We present an $O(n^{1.5}\sqrt{\lg n})$ algorithm for finding the threshold point of an $n \times n$ matrix and an $O(m^{1.5}\lg m)$ algorithm for finding the threshold point of an $m \times n$ matrix (where $m > n$).

### *1.2. Organization of the Paper*

Section 2 reviews the traversal mechanism introduced in the DD paper, establishes its fundamental properties, and shows how it gives rise to the threshold point. Section 3 revisits structural properties of strict saddlepoints used in the subsequent development. Section 4 establishes the relationship between the proposed parameter, pseudo saddlepoints, and strict saddlepoints. Section 5 presents algorithms for computing the threshold point and analyzes their running times. Finally, Section 6 concludes the paper and discusses directions for future work.

## 2. Traversal Mechanism and Threshold Points

The DD paper used a search mechanism to find out whether a value is $v$ the strict saddlepoint or there is no strict saddlepoint in the matrix. This search mechanism is mentioned while discussing a method to search for saddlepoints (or strict saddlepoints as per their definition). Downward Biased Traversal(DBT) and Rightward Biased Traversal(RBT) which we will see later in this section are referred to as horizontal and vertical searches in their paper. We attempt to explore some more properties of this search mechanism. We use a different terminology to help the reader from constantly trying to recollect what horizontal or vertical means. We also felt that the term 'search' used by the authors is a bit misleading, because the search for an element usually ends when you find that particular element which is not true in case of the mechanism described by them.

### *2.1. Traversal Mechanism in a Matrix*

Traversal mechanism. Given a rectangular matrix $M(m \times n)$ and a value $v$, the traversal proceeds as follows.

- The traversal begins from the upper left corner of the matrix $M$, i.e $M[1][1]$.
- If $M[i][j] < v$, then the traversal continues from $M[i][j+1]$.
- If $M[i][j] > v$, then the traversal continues from $M[i+1][j]$.
- The traversal ends if $i = m+1$ or $j = n+1$.
- Whenever $M[i][j] = v$, the tie is resolved in one of two ways.
    1. The traversal that always continues from the next row (i.e $M[i+1][j]$) is called a *Downward Biased Traversal (DBT).*
    2. The traversal that always continues from the next column (i.e $M[i][j+1]$) is called a *Rightward Biased Traversal (RBT).*

**Determining success or failure**:- The DBT is considered to have ended in success, if $i = m+1$, i.e. the traversal exits from the down side of the matrix. Otherwise, it is considered to have ended in failure. Similarly, the RBT is considered to have ended in success, if $j = n+1$, i.e the traversal exits from the right side of the matrix. Otherwise, it is considered to have


Figure 1: Trajectories of DBT and RBT on a matrix with value 55 and 76.

ended in failure. In Figure 1, the DBT and RBT performed with value 55 have ended in success, whereas in case of the value 76, the DBT ends in failure while RBT ends in success.

A **Threshold Point** of a matrix $M$ is defined as a value $v$, such that $DBT(M, v)$ and $RBT(M, v)$ both end in success. In the example shown, the value 55 is the threshold point. The DD paper briefly mentions the existence of such a value in a footnote but does not investigate its properties.

Since the traversal mechanism does not allow the row or column index to decrease, the entries examined by DBT/RBT form a staircase-like trajectory. Let $Traj_{DBT}(M, v)$ (or $Traj_{RBT}(M, v)$) denote the trajectory traced if one performs DBT (RBT) in $M$ with value $v$. This trajectory is a sequence of array indices which are inspected while performing a traversal.

We define an ordering on the trajectories of two traversals corresponding to the distinct values $u$ and $v$. Suppose the first $i$ indices in both trajectories are the same, but the $(i+1)^{\text{th}}$ index in the traversal of $u$ differs from the

$(i+1)^{\text{th}}$ index in the traversal of $v$. If the last common index in both the traversals is $M[x][y]$ then clearly, the subsequent indices in the two trajectories would be $M[x+1][y]$ and $M[x][y+1]$. Without loss of generality, let $M[x+1][y]$ and $M[x][y+1]$ belong to the Trajectories of $u$ and $v$ respectively. In such a situation, we define the ordering of trajectories to be $Traj(M,u) < Traj(M,v)$. We choose to drop the subscript if both the trajectories are of the same type of traversal. Clearly, the first index is common to all trajectories, since every traversal starts from $M[1][1]$.

*2.2. Structural properties of Traversal Mechanism*

**Lemma 2.1.** *If $v$ is the smallest value in $M$, then $\forall v' \leq v$, $DBT(M,v')$ ends in success.*

*Proof.* The traversal starts from $M[1][1]$ for the value $v$. Since $v$ is the smallest value in $M$, the traversal never encounters a value smaller than $v$ and thus the traversal never moves to the second column and keeps moving downwards. Therefore, after examining all the entries of the first column $i$ is set to $m+1$ and the traversal exits from the last row causing DBT to end in success. The traversal has the same trajectory for all values $v' < v$, namely the trajectory that encounters only the entries in the first column. □

Analogously, we can prove

**Lemma 2.2.** *If $v$ is the largest value in $M$, then $\forall v' \geq v, RBT(M,v')$ ends in success.*

**Lemma 2.3.** $Traj_{DBT}(M,u) \leq Traj_{RBT}(M,u)$

*Proof.* If $u$ is not encountered in the $Traj_{DBT}(M,u)$, then it implies that all the values encountered in the traversal are strictly smaller or strictly larger than $u$. In such a situation, $Traj_{DBT}(M,u) = Traj_{RBT}(M,u)$. Otherwise, if the DBT encounters the value $u$ for the first time, then up to that point the DBT and RBT have followed the same trajectory. As soon as the traversal encounters the first instance of $u$, the DBT proceeds the traversal from the subsequent row, while the RBT proceeds the traversal from the subsequent column, proving the claim. □

**Lemma 2.4.** *If $u < v$, then $Traj_{DBT}(M,u) \leq Traj_{DBT}(M,v)$*

*Proof.* As long as the values encountered during the traversals are smaller than $u$ or larger than $v$, both traversals follow the same trajectory. Let $w = M[i][j]$ be the first value encountered during the traversal of $u$ that belongs to the interval $[u, v)$. In such a situation, the traversals for $u$ and $v$ continue from $M[i+1][j]$ and $M[i][j+1]$, respectively. By definition of ordering of trajectories, we conclude that $Traj_{DBT}(M, u) < Traj_{DBT}(M, v)$. Otherwise, if the traversal of both $u$ and $v$, never encounter a value $w$ such that $w \in [u, v)$, then both the traversals would have the same trajectory. □

Analogously, we can prove

**Lemma 2.5.** *If $u < v$, then $Traj_{RBT}(M, u) \leq Traj_{RBT}(M, v)$*

**Corollary 2.6.** *If $DBT(M, v)$ succeeds, then $\forall u < v$, $DBT(M, u)$ also succeeds.*

**Corollary 2.7.** *If $DBT(M, u)$ fails, then $\forall v > u$, $DBT(M, v)$ also fails.*

Lemma shows that Trajectory of DBT with $v$ in a sense is covered by Trajectory of RBT with the same value $v$. Lemmas 2.4 and 2.5 ensure that if $u < v$, then the trajectory of $u$ covers the trajectory of $v$.

**Lemma 2.8.** *If $DBT(M, u)$ succeeds, but its trajectory does not encounter the value $u$, then there exists another value $u' > u$ such that $DBT(M, u')$ also succeeds and has the exact same trajectory as $DBT(M, u)$.*

*Proof.* Since $DBT(M, u)$ ends in success and the trajectory does not contain $u$, it implies that in each row, the trajectory encountered a value that is larger than $u$. Find the smallest element larger than $u$ in the trajectory of $DBT(M, u)$, say $u'$. If we perform $DBT(M, u')$, it will also end in success as it would also have the same trajectory as $DBT(M, u)$. □

**Lemma 2.9.** *If $t < t'$ and there exists no element between $t$ and $t'$ in $M$, then $Traj_{RBT}(M, t) = Traj_{DBT}(M, t')$*

*Proof.* Since there are no elements between $t$ and $t'$, all elements in $M$ other than $t$ and $t'$ are either smaller than both of them or larger than both of them. Thus, if $RBT(M, t)$ does not encounter $t$ or $t'$, then trajectory of $RBT(M, t)$ is the same as the trajectory of $DBT(M, t')$. If $RBT(M, t)$ encounters $t$, then since it is rightward biased it proceeds to the next column, exactly as $DBT(M, t')$ does. Similarly, if $RBT(M, t)$ encounters $t'$, it proceeds to the next row, exactly as $DBT(M, t')$ does. □

*2.3. Existence, Uniqueness, and Characterizations of Threshold Points*

Having established the basic properties of DBT and RBT, we now investigate threshold points. We first prove that every matrix has exactly one threshold point and then derive several useful characterizations of it.

**Lemma 2.10.** *There exists at most one value $v$ such that $RBT(M, v)$ and $DBT(M, v)$ both succeed.*

*Proof.* If all the entries in the matrix are the same (say $s$), then that entry definitely is a value for which both the traversals succeed. For all values smaller than $s$, the RBT would end in failure and for all values greater than $s$, the DBT would end in failure.

Otherwise, if there are at least two distinct entries in $M$, then by Lemmas 2.1 and 2.2, there exist two values, namely $m_{min}$ and $m_{max}$ in the matrix, for which $DBT(M, m_{min})$ and $RBT(M, m_{max})$ end in success, where $m_{min}$ and $m_{max}$ are the smallest and largest values in $M$, respectively.

If $x$ does not occur in $M$, then $DBT(M, x)$ and $RBT(M, x)$ have the same trajectory, and thus one of the traversals ends in failure. Thus, the values which do not exist in $M$ cannot be the value of threshold point.

Now, we focus our attention to only those values which are present in the matrix $M$. Suppose $t$ is the smallest value in $M$ for which both $DBT(M, t)$ and $RBT(M, t)$ succeed. Let $t'$ be the smallest value in $M$ which is larger than $t$.

By Lemma 2.9, $DBT(M, t')$ and $RBT(M, t)$ have the exact same trajectory. Since $RBT(M, t)$ exits with $j = n + 1$, $DBT(M, t')$ also terminates with $j = n + 1$, causing $DBT(M, t')$ to end in failure. By Corollary 2, we can also infer that $DBT(M, t'')$ would also fail for all values $t'' > t'$.

Thus, there can be at most one value for which DBT and RBT terminate in success. □

Based on the proof of Lemma 2.10, we can give an alternate characterization of a threshold point.

**Corollary 2.11.** *A **threshold point** of a matrix $M$ is a value $t$ that is present in $M$ such that $DBT(M, t)$ succeeds, but $DBT(M, t')$ ends in failure for all $t' > t$.*

If $m_{min}$ and $m_{max}$ denote the smallest and largest values in $M$, then there exists $t \in [m_{min}, m_{max}]$, such that the $DBT$ function acts like a step function which succeeds for all values $\leq t$ and fails for all values $> t$.

**Lemma 2.12.** *Every matrix has a value $v$, such that $DBT(M, v)$ and $RBT(M, v)$ end in success.*

*Proof.* By Lemma 2.1, we know that $DBT(M, m_{min})$ succeeds. Let $t$ be the largest value for which $DBT(M, t)$ succeeds. If $RBT(M, t)$ ends in success, then both $DBT(M, t)$ and $RBT(M, t)$ end in success, and hence $t$ is a threshold point.

For the sake of argument, suppose we perform $RBT(M, t)$ and it ends in failure, i.e it exits with $i = m + 1$. This implies that the traversal encounters a value in each row that is strictly larger than $t$. Let $z$ be the smallest value encountered during the traversal that is strictly larger than $t$. This would imply that $DBT(M, z)$ also has the same trajectory as $RBT(M, t)$ and thus $DBT(M, z)$ is a success. But this contradicts our assumption that $t$ is the largest value for which DBT ends in success. Therefore, $RBT(M, t)$ also ends in success and the value $t$ is a threshold point. □

Combining Lemmas 2.10 and 2.12 immediately yields the following theorem.

**Theorem 2.13.** *Every matrix $M$ has exactly one threshold point.*

**Lemma 2.14.** *If $v$ is the threshold point, then both $DBT(M, v)$ and $RBT(M, v)$ encounter an index $[i][j]$ such that $M[i][j] = v$.*

*Proof.* If $v$ is the value of the threshold point, then until the trajectories of $DBT(M, v)$ and $RBT(M, v)$ do not encounter the value $v$, their trajectories are the same. Their entire trajectories cannot be the same, since otherwise one of them would end in failure. Thus, there is at least one index common to both these trajectories where they encounter the value $v$. □

Let *last common threshold location* denote the location $M[x][y]$, such that $M[x][y]$ is the last common index occurring in both $Traj_{DBT}(M, v)$ and $Traj_{RBT}(M, v)$.

This last common index must have value $v$. Otherwise, if the last common index had a value other than $v$, then the subsequent index inspected by both trajectories would also be the same, contradicting the assumption that it is the last common index.

Now that we have established what a threshold point is, we would like to highlight the fact that the algorithm defined in Lemma 3 of the DD paper, which when given a value $s$ tests whether $s$ is a strict saddlepoint or

not is in essence trying to test whether $s$ is a threshold point or not. The final verification step of that algorithm relies on the last common threshold location.

## 3. Structural Properties of Strict Saddlepoints

This section recalls several properties of strict saddlepoints that will be used in the subsequent comparison with threshold points. Although these results are well known or follow directly from the definition of strict saddlepoints, we include short proofs for completeness because they are not readily available in the literature in the form required here.

**Lemma 3.1.** *If $s$ is the value of the strict saddlepoint of $M$ in row $i$, then all the rows (except row $i$) of $M$ has a value which is greater than $s$.*

*Proof.* Let $r \neq i$ be another row, and let $s_1$ denote a maximum element of row $r$. Let $z$ denote the entry in row $r$ and the column containing the strict saddlepoint $s$.

Since $s$ is the unique minimum of its column,$s < z$. Since $s_1$ is a maximum element of row $r$, $z \leq s_1$. Hence,

$$s < s_1.$$

□

Analogously, we can prove

**Lemma 3.2.** *If $s$ is the value of the strict saddlepoint of $M$ in column $j$, then all the columns (except column $j$) of $M$ has a value which is smaller than $s$.*

The previous lemmas describe structural properties of a strict saddlepoint once it is known to exist. The following two lemmas provide useful bounds on the value of a strict saddlepoint and will play an important role in relating strict saddlepoints to threshold points.

**Lemma 3.3.** *If every row of $M$ has a value which is greater than or equal to $x$, then if the strict saddlepoint exists has a value which is greater than or equal to $x$.*

*Proof.* Since each row has a value greater than or equal to $x$, the row which contains the strict saddlepoint also has a value which is greater than or equal to $x$. Since the strict saddlepoint is the maximum element of its row, its value will also be greater than or equal to $x$. □

Analogously, we can prove

**Lemma 3.4.** *If every column of $M$ has a value which is less than or equal to $x$, then if the strict saddlepoint exists has a value which is less than or equal to $x$.*

**Lemma 3.5.** *A strict saddlepoint if it exists is unique by location.*

*Proof.* Suppose, for contradiction, that $x = M[i][j]$ and $y = M[p][q]$ be two strict saddlepoints of $M$ at two distinct locations. Clearly, $x$ and $y$ do not belong to the same row, otherwise they would not be the unique maximum of their rows. Similarly, they cannot belong to the same column. Let $z_1 = M[i][q]$ and $z_2 = M[p][j]$. Based on the definition of strict saddlepoints, $x > z_1$ and $z_1 > y$. Similarly, using $z_2$ we can conclude that $x < z_2$ and $z_2 < y$. Using $z_1$ and $z_2$, we can conclude that $x < y$ and $x > y$, which leads to a contradiction. Thus, a strict saddlepoint cannot be present at two locations. □

The following invariance property will later be contrasted with the corresponding behaviour of threshold points.

**Lemma 3.6.** *If a matrix has a strict saddlepoint, then its value is invariant under row and column permutations.*

*Proof.* Let $x$ be the strict saddlepoint of $M$. A row permutation preserves the entries within each row, so $x$ remains the unique maximum of its row. Since column membership is unchanged, $x$ also remains the unique minimum of its column. Hence, $x$ continues to be the strict saddlepoint after any row permutation. The proof for column permutations is analogous. □

The preceding results provide the structural properties of strict saddlepoints needed in the remainder of the paper. We now establish their relationship with threshold points.

## 4. Relationship between Threshold points and Strict Saddlepoints

This section focuses on understanding the similarities and differences between a threshold point and a strict saddlepoint.

*4.1. Similarities between Threshold Points and Strict Saddlepoints*

**Theorem 4.1.** *If $M$ has a strict saddlepoint, then the threshold point and the strict saddlepoint have the same value.*

*Proof.* Since $t$ is the value of the threshold point of $M$, we know that $DBT(M, t)$ and $RBT(M, t)$ both end in success. This implies that every row has a value greater than or equal to $t$ and every column has a value less than or equal to $t$. By Lemma 3.3 and 3.4, we can conclude that the value of SSP is greater than or equal to $t$ and simultaneously less than or equal to $t$. Thus, the only valid value of SSP if it exists is $t$.

□

For the sake of completeness, we state the following lemma from the DD paper [1].

**Lemma 4.2.** *[1] Given an $m \times n$ matrix $A$ and a value $s$, we can find, in time $O(m + n)$, a SSP of $A$ of value $s$, or report that no such SSP exists.*

Based on the definition of threshold points, the proof of the aforementioned lemma in the DD paper can be viewed to have three phases. The first phase involves checking whether $s$ is a threshold point or not. The second phase involves locating the last threshold point which is present at say $A[i][j]$ and the third phase consists of verifying whether $A[i][j]$ is the unique maximum in row $i$ and unique minimum in column $j$.

In first phase, if both the traversals end in success, the algorithm moves onto the next phase otherwise, it claims that $s$ is not the threshold point and thus not the value of the SSP.

If the first phase succeeds, then we enter the second phase which involves locating an instance of the value $s$, which can contain the potential SSP. The DD paper shows that if $s$ is the value of the SSP, then it will be encountered while performing horizontal and vertical searches (i.e DBT and RBT). They also claim that if $RBT(A, s)$ exits from row $i$ and $DBT(A, s)$ exits from column $j$, then a copy of $s$ exists at $A[i][j]$ and values present at any index except $A[i][j]$ cannot be the SSP.

Then, the third phase verifies whether the value at $A[i][j]$ is the unique maxmimum in row $i$ and unique minimum in column $j$.

The description of the second phase contains a minor inaccuracy. However, this does not affect the correctness or running time of the algorithm. They make an assumption which is not always true, however fortunately that

assumption is not relevant to the correctness of the lemma. Their assumption that if the two searches succeed and exit from row $i$ and column $j$, then $s = A[i][j]$ is not always true. This assumption is true if $A$ has $s$ to be the value of its SSP, otherwise $A[i][j]$ may not be equal to $s$.

The following counter-example shows that it is not always true. In the following matrix, the two types of searches succeed with value 3 and they exit from $2^{nd}$ row and $1^{st}$ column, but $A[2][1] = 5 \neq 3$.

$$\begin{bmatrix} 3 & 1 & 4 \\ 5 & - & 2 \\ 6 & - & - \end{bmatrix}$$

The following modification eliminates this issue without affecting the correctness or running time of the algorithm. If both the searches succeed it implies, that the two searches diverge at some point. The traversals may encounter the value of threshold point multiple times. By Lemma 2.14, we know that there exists a last common threshold location. One can then perform phase 3 from the location of last common threshold location.

**Theorem 4.3.** *If $M$ has a strict saddlepoint, then the last threshold point coincides with the strict saddlepoint.*

*Proof.* Let $t$ be the value of the SSP and let the SSP be at location $M[p][q]$. By Theorem 4.1, we know that the value of threshold point is also $t$. We observe the trajectories of $DBT(M, t)$ and $RBT(M, t)$.

If $p = q = 1$, then the unique maximum of first row and the unique minimum of first column is at $M[1][1]$. All other values of first row are smaller than $t$ and all other values of first column are larger than $t$. DBT and RBT would both succeed with value $t$ and they would exit from first column and first row respectively and both result in success. Since, $M[1][1]$ would the first and last common location in the RBT and DBT trajectories, $M[1][1]$ is the last threshold point in this case.

If $p = 1$ and $q \neq 1$, then the RBT would exit from the first row and DBT would exit from column $q$. Both the traversals would be probing all the entries in the first row until they hit the location of SSP which is value $t$ and then they diverge. Thus, the last threshold point in this case would also be the location of the SSP.

Analogously, one can also prove the case when $p \neq 1$ and $q = 1$.

If $p, q \neq 1$, we analyse the trajectory of $DBT(M, t)$ and $RBT(M, t)$. The traversals would either reach column $q$ or reach row $p$ before they ter-

minate. If DBT/RBT reaches row $p$ before it reaches column $q$ and since $M[p][q]$ is the unique maximum of row $p$, the DBT/RBT rules would force the traversal towards $M[p][q]$, since all the value except $M[p][q]$ are smaller than $t$. Similarly, if the DBT/RBT reaches column $q$ before it reaches row $r$, the DBT/RBT rules would force the traversal towards $M[p][q]$. Thus, both $DBT(M, t)$ and $RBT(M, t)$ would reach location $M[p][q]$. After this point, DBT would probe rest of entries in column $q$ and exit while $RBT$ would probe rest of the entries in row $p$ and exit. Thus $M[p][q]$ is the last threshold point that both the traversals encounter. □

**Lemma 4.4.** *The threshold point is a pseudo saddlepoint.*

*Proof.* The DD paper defines a pseudo-saddlepoint (PSP) of $M$ as an entry $v$ of $M$ such that every row of $M$ has an entry larger or equal to $v$ and every column of $M$ has an entry smaller or equal to $v$. If $t$ is the value of the threshold point, then clearly $DBT(M, t)$ and $RBT(M, t)$ end in success, which implies that every row has value greater than equal to $t$ and every column has value less than or equal to $t$. □

Thus, if $x$ it the value of the SSP, then $x$ is also the value of the threshold point and if $x$ is the value of threshold point, then $x$ is also a valid pseudo saddlepoint. If SSP exists, then all the three entities have the same value. However, if SSP does not exist, then a threshold point is one of the possibly many pseudo saddlepoints.

*4.2. Differences between Threshold Points and Saddlepoints*

Aside from the fact that threshold point always exists in a matrix, while an SSP may not exist, they are different with regards to some other properties as well. We use a sample matrix to showcase the difference.

$$\begin{bmatrix} 8 & 2 & 4 \\ 9 & 3 & 7 \\ 1 & 6 & 5 \end{bmatrix}$$

In the above matrix $M_1$, one can verify that the threshold point is 6, but the matrix does not have a strict saddlepoint.

1. By Lemma 3.6, we know that the strict saddlepoint of a matrix is preserved under row exchanges and column exchanges. However, the

threshold point of a matrix can be different under such row/column exchanges.

$$\begin{bmatrix} 1 & 6 & 5 \\ 9 & 3 & 7 \\ 8 & 2 & 4 \end{bmatrix}$$

The threshold point of the above matrix $M_2$ is 4. $M_2$ is the matrix obtained by swapping the first and third rows of $M_1$.

2. Most algorithms for finding a strict saddlepoint rely on the observation that if row $i$ (or column $j$) does not contain the strict saddlepoint, then one can instead search for the strict saddlepoint in the submatrix obtained by deleting row $i$ (or column $j$).

$$\begin{bmatrix} 2 & 4 \\ 3 & 7 \\ 6 & 5 \end{bmatrix}$$

If we somehow know that first column of $M_1$ does not contain the threshold and then try to find the threshold point of the matrix obtained from $M_1$ by removing its first column, the value of the threshold point changes to 4.

Thus, due to these aforementioned differences one cannot apply most of the published techniques known for finding the SSP in order to find the threshold point, without any significant modification to the strategy.

## 5. Algorithms to find the threshold point

At the end of the previous section, we observed that existing techniques for finding strict saddlepoints cannot be applied directly to find threshold points. In this section, we present several algorithms for finding the threshold point of an $n \times n$ square matrix. These algorithms can easily be generalized to work for non-square matrices with minor changes in the parameters of the algorithms. Since this is the first paper discussing the topic of threshold points, we intentionally focus on introducing the algorithms for square matrices for the sake of succinctness and simplicity.

First we mention an elementary technique to find the threshold point.

**Theorem 5.1.** *The threshold point of an $m \times n$ matrix can be computed in $O(mn)$ time.*

**Algorithm 1** Finding the Threshold Point Using Binary Search

**Require:** Matrix $M$
**Ensure:** Threshold point of $M$

```
S ← { M[i, j] | 1 ≤ i ≤ m, 1 ≤ j ≤ n }
repeat
    med ← median(S)
    if DBT(M, med) fails then
        S ← {x ∈ S : x < med}
    else if RBT(M, med) fails then
        S ← {x ∈ S : x > med}
    end if
until DBT(M, med) succeeds and RBT(M, med) succeeds
return med
```

*Proof.* Let $S$ denote the collection of all entries of $M$, including duplicate values, and let $N = mn$.

**Correctness-** In every non-terminating iteration, the algorithm eliminates at least half of the candidate values from the candidate set $S$. By Corollaries 2.6 and 2.7, the outcomes of $DBT(M, med)$ and $RBT(M, med)$ determine whether the threshold point is smaller or larger than $med$. Hence, the threshold point is always retained in the candidate set while at least half of the remaining candidates are eliminated in each iteration. The algorithm terminates when both traversals succeed, at which point $med$ is the threshold point.

**Time complexity analysis-** The matrix contains $N = mn$ entries. Since the size of the candidate set $S$ is halved in every non-terminating iteration, the algorithm performs $O(\lg(mn))$ iterations. Each iteration performs one $DBT$ traversal and at most one $RBT$ traversal. Since each traversal requires $O(m + n)$ time, the total time spent in the $DBT$ and $RBT$ traversals is $O((m + n)\lg(mn))$. The running time of the median finding subroutine satisfies the following recurrence relation.

$$T_{med}(N) = O(N) + T_{med}(N/2). \tag{1}$$

Solving the recurrence yields $T_{med}(N) = O(N)$. Since $O((m + n)\lg(mn)) = o(mn)$, the median finding subroutine dominates the running time. Hence, the overall algorithm runs in $O(mn)$ time and reports the threshold point. In case of an $n \times n$ square matrix, this gives an $O(n^2)$ time algorithm. □

*5.1. Recursive algorithm to find the threshold point*

In essence, the basic algorithm requires one to probe all the elements of the matrix. The basic algorithm probes all the entries of the matrix. The recursive algorithm presented below improves upon this approach by tracing the trajectory that DBT or RBT would follow if performed with the threshold point, without knowing the threshold point in advance. The recursive algorithm below assumes that the quantities $m_1$ and $n_1$ have been chosen as functions of the dimensions of initial input matrix. These values remain fixed throughout the recursive execution of the algorithm. Their specific values are not needed for correctness and will be specified later during the running-time analysis.

**Algorithm 2** Threshold Point in $M\,(m \times n)$

```
1:  procedure THRESHOLD-POINT(M)
2:      S ← top-left m_1 × n_1 submatrix of M
3:      min ← minimum entry in the first column of S
4:      max ← maximum entry in the first row of S
5:      if DBT(M, min) fails then
6:          M' ← submatrix obtained by deleting the first m_1 rows of M
7:          return THRESHOLD-POINT(M')
8:      else if RBT(M, max) fails then
9:          M' ← submatrix obtained by deleting the first n_1 columns of M
10:         return THRESHOLD-POINT(M')
11:     else
12:         Find t_low ∈ S such that
              DBT(M, t_low) succeeds, but DBT(M, t') fails
              ∀ t' > t_low occurring in S.
13:         if RBT(M, t_low) succeeds then
14:             return t_low
15:         else
16:             (x, y) ← first index outside S probed by RBT(M, t_low)
17:             M' ← submatrix obtained by deleting the first x − 1 rows
                        and the first y − 1 columns of M.
18:             return THRESHOLD-POINT(M')
19:         end if
20:     end if
21: end procedure
```

**Correctness**

Let $t$ be the value of the threshold point in $M$. We show that every recursive call is made on a submatrix whose threshold point is also $t$. Consequently, when the recursion terminates, the algorithm reports the threshold point of the original matrix. The recursion continues while the current submatrix contains at least $m_1$ rows and $n_1$ columns. For simplicity of presentation, the pseudocode assumes that the current submatrix has at least $m_1$ rows and $n_1$ columns. When either dimension becomes smaller, the threshold point of the remaining submatrix is obtained using the basic algorithm of Theorem 5.1. As observed earlier, the trajectories of $DBT(M, t)$ and $RBT(M, t)$ coincide until they encounter the first entry whose value is $t$.

In each recursive step, the algorithm focuses on a top-left submatrix $S$ of dimension $m_1 \times n_1$. Line 5 checks for the possibility that $t < min$. Since every entry in the first column of $S$ is at least $min > t$, the traversal $DBT(M, t)$ and $RBT(M, t)$ necessarily moves down the first column through the first $m_1$ rows before entering row $m_1 + 1$. Consequently, the remainder of the traversal depends only on the submatrix obtained by deleting the topmost $m_1$ rows of $M$, whose threshold point is also $t$. Thus, the recursive call in line 7 preserves the threshold point.

Similarly, line 8 checks for the possibility when $t > max$. Since every entry in the first row of $S$ is at most $max < t$, the traversal $DBT(M, t)$ and $RBT(M, t)$ necessarily moves right along the first row through the first $n_1$ columns before entering column $n_1 + 1$. Consequently, the remainder of the traversal depends only on the submatrix obtained by deleting the leftmost $n_1$ columns of $M$, whose threshold point is also $t$. Thus, the recursive call in line 10 preserves the threshold point.

The else case at line 11 is encountered if $min < t < max$. At this point, either $S$ contains a copy of the threshold point or it does not. Line 12 probes all the entries of $S$ and applies a binary search, similar to the algorithm in Theorem 5.1, to find the largest value $t_{low} \in S$ such that $DBT(M, t_{low})$ succeeds. This binary search is valid because of the monotonicity established in Corollaries 2.6 and 2.7. Thus, line 12 correctly identifies $t_{low}$.

If $RBT(M, t_{low})$ also succeeds, then by definition $t_{low}$ is the threshold point, and the algorithm returns it in line 14.

Otherwise, $RBT(M, t_{low})$ fails and the algorithm executes the recursive call in line 18. We now show that this recursive call also preserves the

threshold point. To do so, we compare the trajectories of $DBT(M, t_{low})$ and $DBT(M, t)$ and show that they coincide until reaching the first index outside $S$.

Since $t_{low} < t$, there are three possible cases for the values of the entries encountered during traversal $DBT(M, t_{low})$ until we reach the index $(x, y)$.

1. The element encountered is less than or equal to $t_{low}$.
   In such a situation, the traversals with value $t$ and $t_{low}$ would both continue the traversal from the next column.
2. The element encountered is greater than $t$.
   In such a situation, the traversals with value $t$ and $t_{low}$ would both continue the traversal from the next row.
3. The element encountered is in the range $(t_{low}, t]$
   This case cannot happen, otherwise it would contradict the property of $t_{low}$, that it is the largest value smaller than $t$ whose DBT ends in success.

Thus, the trajectories of both $DBT(M, t_{low})$ and $DBT(M, t)$ would coincide until they reach $M[x][y]$ which is the first index outside $S$.

An analogous argument shows that $RBT(M, t)$ also reaches the same first index $(x, y)$ outside $S$. Since $t_{low} < t$ and no entry encountered before reaching $(x, y)$ has a value in the interval $(t_{low}, t]$, the decisions made by $RBT(M, t_{low})$ and $RBT(M, t)$ are identical until $(x, y)$ is reached.

Consequently, the remainder of the traversal depends only on the submatrix $M'$ described at line 17. Therefore, $M'$ has the same threshold point $t$, and the recursive call preserves the threshold point.

**Time complexity analysis**

First, we provide the running time analysis for arbitrary values of $m_1$ and $n_1$.

There are $mn$ entries in the matrix. Each recursive step begins by examining the top-left $m_1 \times n_1$ submatrix. In every recursive step, while going from solving the problem on $M$ to solving the problem on $M'$ we are either discarding $m_1$ rows or $n_1$ columns.

Lines 2,3 and 4, collectively require $O(m_1 n_1 + m_1 + n_1)$ time. Since each DBT and RBT scan requires $O(m + n)$ time, the branch at lines 5–7 or the branch at lines 8–10 incurs $O(m + n)$ cost each.

Line 12, requires us to examine all entries of $S$ which takes $O(m_1 n_1)$ time and then we need to perform $O(\lg m_1 n_1)$ number of DBT searches to find

$t_{low}$ which would in total take $O(m_1n_1 + (m+n)\lg m_1n_1)$. Line 13, performs an RBT scan and thus requires $O(m+n)$ time. The final else case at lines 15 through 18, simply discards $x-1$ rows and $y-1$ columns requires $O(1)$ time.

Thus, each recursive step requires

$$O((m_1n_1 + m_1 + n_1) + (m+n) + (m_1n_1) + (m+n)\lg(m_1n_1))$$

which simplifies to

$$O(m_1n_1 + (m+n)\lg(m_1n_1))$$

time.

Since each recursive step removes either $m_1$ rows or $n_1$ columns, there can be at most $m/m_1$ row deletions and at most $n/n_1$ column deletions. Hence the total number of recursive steps is

$$O\left(\frac{m}{m_1} + \frac{n}{n_1}\right).$$

Finally, when either the number of rows becomes smaller than $m_1$ or the number of columns becomes smaller than $n_1$, the recursion terminates and the basic algorithm of Theorem 5.1 is invoked. That algorithm runs in $O(mn_1 + m_1n)$. This cost is incurred at most once.

Therefore, the running time of the algorithm is

$$T(M) = O\left[\left(m_1n_1 + (m+n)\lg m_1n_1\right) * \left(\frac{m}{m_1} + \frac{n}{n_1}\right)\right] + O(mn_1 + m_1n) \quad (2)$$

**Rectangular matrices.** For rectangular matrices, assume without loss of generality that $m \geq n$, and choose $m_1 = \sqrt{m}$ and $n_1 = \sqrt{n}$. Substituting these values into the running-time expression, we obtain

$$T(M) = O\left[\left(\sqrt{mn} + (m+n)\lg\sqrt{mn}\right)\left(\sqrt{m} + \sqrt{n}\right)\right] + O(m\sqrt{n} + n\sqrt{m}).$$

Since $m \geq n$, we have

$$\sqrt{mn} \leq m, \qquad m+n \leq 2m, \qquad \sqrt{m}+\sqrt{n} \leq 2\sqrt{m}, \qquad m\sqrt{n}+n\sqrt{m} \leq 2m\sqrt{m}.$$

Hence,

$$\begin{aligned} T(M) &= O\left[(m + m\lg m)\sqrt{m}\right] + O(m\sqrt{m}) \\ &= O(m^{1.5}\lg m). \end{aligned}$$

Thus, the recursive algorithm runs in $O(m^{1.5} \lg m)$ time for an $m \times n$ matrix. Whenever $\sqrt{m} \lg m = o(n)$, this improves upon the $O(mn)$ running time of Theorem 5.1.

**Square matrices.** For $n \times n$ square matrices, choose $m_1 = n_1 = \sqrt{n \lg n}$. Substituting these values into the running-time expression and using $\lg(n \lg n) = \Theta(\lg n)$, we obtain

$$\begin{aligned} T(M) &= O\left[\left(m_1 n_1 + (m+n)\lg m_1 n_1\right)\left(\frac{m}{m_1} + \frac{n}{n_1}\right)\right] + O(mn_1 + m_1 n) \\ &= O\left[\left(n \lg n + 2n \lg(n \lg n)\right)\left(\frac{\sqrt{n}}{\sqrt{\lg n}} + \frac{\sqrt{n}}{\sqrt{\lg n}}\right)\right] + O\left(2n\sqrt{n \lg n}\right) \\ &= O\left(n\sqrt{n \lg n}\right). \end{aligned}$$

Thus, for square matrices, the recursive algorithm runs in $O(n^{1.5}\sqrt{\lg n})$ time, improving upon the $O(n^2)$ running time of Theorem 5.1.

**Theorem 5.2.** *For an $m \times n$ matrix with $m \geq n$, the threshold point can be reported in $O(m^{1.5} \lg m)$ time.*

**Theorem 5.3.** *For an $n \times n$ matrix, the threshold point can be reported in $O(n^{1.5}\sqrt{\lg n})$ time.*

### *5.2. Simple algorithm for matrices whose entries belong to a fixed-size integer universe*

Suppose the entries of the matrix $M$ belong to the integer universe $U = \{0, 1, \ldots, u-1\}$. By Corollary 2.6, the values for which $DBT(M, t)$ succeeds form a prefix of the universe. Hence, the threshold point can be found by performing a binary search over the universe to determine the largest value for which $DBT$ succeeds. Since each iteration performs one $DBT$ traversal, the algorithm requires $O(\lg u)$ iterations, each taking $O(m + n)$ time.

**Theorem 5.4.** *For an $m \times n$ matrix whose entries belong to the universe $U = \{0, 1, \ldots, u-1\}$, the threshold point can be reported in $O((m+n) \lg u)$ time.*

Once the threshold point has been found, the last common threshold location can also be reported in $O(m + n)$ time using the lemma from the DD paper.

## 6. Future Work

Future work includes proving tighter lower bounds for the threshold-point problem, whose best known lower bound is currently the trivial $\Omega(m+n)$. It would also be interesting to improve the $O(m^{1.5}\lg m)$ and $O(n^{1.5}\sqrt{\lg n})$ upper bounds established in this paper. Another promising direction is the development of randomized algorithms for finding the threshold point. Finally, threshold points represent one type of pseudo saddlepoint; it may be worthwhile to investigate other notions of pseudo saddlepoints and their algorithmic properties.

## References


[1] J. Dallant, F. Haagensen, R. Jacob, L. Kozma, S. Wild, Finding the saddlepoint faster than sorting, in: M. Parter, S. Pettie (Eds.), 2024 Symposium on Simplicity in Algorithms, SOSA 2024, Alexandria, VA, USA, January 8-10, 2024, SIAM, 2024, pp. 168–178.

[2] D. E. Knuth, The Art of Computer Programming, Volume 1: Fundamental Algorithms, 1st Edition, Addison-Wesley Publishing Company, Reading, Massachusetts, 1968.

[3] D. C. Llewellyn, C. Tovey, M. Trick, Finding saddlepoints of two-person, zero sum games, Am. Math. Monthly 95 (10) (1988) 912–918. doi:10.2307/2322384.

[4] D. Bienstock, F. Chung, M. L. Fredman, A. A. Schäffer, P. W. Shor, S. Suri, A note on finding a strict saddlepoint, The American Mathematical Monthly 98 (5) (1991) 418–419.

[5] J. Dallant, K. G. L. Haagensen, R. Jacob, L. Kozma, S. Wild, An optimal randomized algorithm for finding the saddlepoint, in: 32nd Annual European Symposium on Algorithms (ESA 2024), Vol. 308 of LIPIcs, 2024, pp. 44:1–44:16.

[6] E. O. Thorp, The probability that a matrix has a saddle point, Information Sciences 19 (2) (1979) 91–95.

[7] M. Hofri, On the distribution of a saddle point value in a random matrix, Tech. rep., Worcester Polytechnic Institute (WPI) (2006).